Warm up

Assume the string is less than 100 characters long. What does the following function (attempt to) do? Can you spot any bugs?

```
#include <ctype.h>
char* mystery(char* string) {
    char res[100];
    size_t i = 0;
    while (string[i]) {
        res[i] = toupper(string[i]);
        i++;
    }
    return res;
}
```

Warm up

Assume the string is less than 100 characters long. What does the following function (attempt to) do? Can you spot any bugs?

```
#include <ctype.h>
char* mystery(char* string) {
    char res[100];
    size_t i = 0;
    while (string[i]) {
        res[i] = toupper(string[i]);
        i++;
    }
    return res;
}
```

- Attempts to convert `string` to uppercase and return the result
- But:
  - Missing NUL terminator for the result string
  - `res` is a local array (allocated on the stack), so it will be deallocated when the function returns

```
warning: address of stack memory associated with local variable 'res' returned
      [-Wreturn-stack-address]
```

Warm up

Corrected version of `mystery`:

```c
#include <ctype.h>
#include <stdlib.h>

char* mystery(char* string) {
    char* res;
    size_t i = 0;

    res = malloc(100 * sizeof(char));
    if (res == NULL) { /* handle malloc failure */ }

    while (string[i]) {
        res[i] = toupper(string[i]);
        i++;
    }
    res[i] = '\0'; /* add NUL terminator */
    return res;
}
```

Precept 9: Debugging Dynamic Memory Management

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

February 26, 2026

Overview

Allocating memory on the heap

- Dynamically allocated memory is allocated on the *heap*
  - Can be allocated and deallocated at runtime
  - Can be larger than the stack
  - Can still be valid after the function that allocated it returns (unlike stack-based memory)
  - Must be managed manually by the programmer
- `malloc`: allocates a block of memory of a specified size

```
void *malloc(size_t size);
```

- `calloc`: allocates a block of memory for an array of elements, initialises each element to zero

```
void *calloc(size_t count, size_t size);
```

- `realloc`: changes the size of a previously allocated block of memory

```
void *realloc(void *ptr, size_t size);
```

Deallocating memory

- `free` is used to deallocate dynamically allocated memory

```
void free(void *ptr);
```

- `ptr` is a pointer to a block of memory previously allocated by `malloc`, `calloc` or `realloc`
- One call to `free` for *every* call to `malloc`, `calloc` or `realloc`
- What if we:
  - forget to `free` a pointer?
  - call `free` on a pointer that was not allocated by `malloc`, `calloc` or `realloc`?
  - call `free` on a pointer that has already been `free`'d?

Overview

What can go wrong?

- Forgetting to `free` memory → memory leaks
- Calling free on a pointer that was not allocated by `malloc`, `calloc` or `realloc` → invalid free
- Calling free on a pointer that has already been `free`'d → double free
- Using a pointer after it has been `free`'d → dangling pointers
- Improper reallocation → memory leaks, dangling pointers, etc.

Memory leaks

- Memory leaks occur when we forget to `free` heap-allocated memory after use
- Can lead to increased memory usage (lower performance)
- Program behaves reasonably, but may eventually run out of memory
- Difficult to detect, especially if the program is not running for a long time or if the leak is small
- Common in long-running programs (e.g. servers, databases) where memory usage can grow over time
- Tools exist (e.g., meminfo, valgrind) that can help you to detect memory leaks (later!)

Dangling pointers

- Dangling pointers point to memory that has been deallocated (i.e., `free`'d or stack memory that has gone out of scope)
- Using a dangling pointer (e.g., dereferencing it) is undefined behaviour

```
int* p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p); /* undefined, garbage output */
```

- Mitigation: set pointer to `NULL` after freeing it

```
int* p = malloc(sizeof(int));
*p = 42;
free(p);
p = NULL; /* mitigate dangling pointer */
printf("%d\n", *p); /* still undefined, but crashes */
```

  - `free`'ing a `NULL` pointer is safe (no-op)
  - Dereferencing a `NULL` pointer will typically cause a crash, which is easier to debug (can use gdb!)
  - Does not address pointers to out-of-scope stack memory

Invalid and double free

- Invalid free: calling `free` on a pointer that was not allocated by `malloc`, `calloc` or `realloc`

```
int x;
free(&x); /* invalid free */
```

- Double free: calling `free` on a pointer that has already been `free`'d

```
int* p = malloc(sizeof(int));
free(p);
free(p); /* double free */
```

  - Mitigation: again, set pointer to `NULL` after freeing it
- Both are undefined behaviour
  - Program may crash or exhibit other unexpected behaviour
  - System-dependent behaviour

Improper reallocation

- `realloc` returns a pointer to a new block of memory that is at least `size` bytes in size
  - `void *realloc(void *ptr, size_t size);`
  - Contents of the old block are copied to the new block
- Does this work?

```
int* arr = malloc(NUM_ELTS * sizeof(int));
/* ... use arr ... */

realloc(arr, NEW_NUM_ELTS * sizeof(int));
```

- Problem 1: What if the new block of memory allocated by `realloc` is at a different address than the original block?
- Problem 2: What if `realloc` fails to allocate the new block?
  - `realloc` returns `NULL`→ should check for this
- Solutions?

Reallocation, the correct way

- Solution:
  - Use a temporary pointer to store the result of `realloc`
  - Check if temporary pointer is `NULL` before updating original pointer
  - If `realloc` fails, original pointer is still valid (and can still be `free`'d)

```
int* arr = malloc(NUM_ELTS * sizeof(int));
/* ... use arr ... */

int* tmp = realloc(arr, NEW_NUM_ELTS * sizeof(int));
if (tmp == NULL) {
    /* handle realloc failure */
} else {
    arr = tmp;
}
```

- Food for thought: is this efficient?
  - Not really: in the worst case, we may have to copy a possibly large array to a new location in memory
  - Alternatives? Allocating maximum array size to begin with

Overview

meminfo

- `meminfo` is a COS 217-specific tool that can help you to detect simple memory leaks and corruption
- Usage
  - Compile your program using `gcc217m`: `gcc217m prog.c -o prog`
  - Run your program normally: `./prog`
  - Find the `meminfoXXXX.out` file in the current directory with `ls meminfo*.out`
  - Generate a summary report: `meminforeport meminfoXXXX.out`
  - Delete the `meminfoXXXX.out` file when done: `rm meminfoXXXX.out`
- Total is the difference between the total number of bytes allocated and the total number of bytes freed
  - Ideally should be $= 0$ (no memory leaks or double frees)
  - If > 0, more bytes were allocated than freed (e.g., memory leak)
  - If < 0, more bytes were freed than allocated (e.g., double free)
- Source code available in `/u/cos217/bin/ARM/meminfo` on armlab

Examining meminfo output

```
$ meminforeport meminfo3306435.out
Errors:
Summary Statistics:
   Maximum bytes allocated at once: 64
   Total number of allocated bytes: 112
Statistics by Line:
          Bytes   Location
            -64   sort4.c:112
             48   sort4.c:23
             16   sort4.c:76
              0   TOTAL
Statistics by Compilation Unit:
              0   sort4.c
              0   TOTAL


$ meminforeport meminfo3306646.out
Errors:
   ** 64 un-freed bytes (1 block) allocated at sort5.c:23
Summary Statistics:
   Maximum bytes allocated at once: 64
   Total number of allocated bytes: 112
Statistics by Line:
          Bytes   Location
             48   sort5.c:23
             16   sort5.c:76
             64   TOTAL
Statistics by Compilation Unit:
             64   sort5.c
             64   TOTAL
```

valgrind

1.1. How do you pronounce "Valgrind"?

The "Val" as in the word "value". The "grind" is pronounced with a short 'i' -- ie. "grinned" (rhymes with "tinned") rather than "grined" (rhymes with "find").

Don't feel bad: almost everyone gets it wrong at first.

Source: `https://valgrind.org/docs/manual/faq.html#faq.pronounce`

- `valgrind` is a tool available on Linux to detect memory leaks, invalid memory access and other memory errors
- Usage
  - Compile your program with debugging symbols (`-g` flag in gcc)
  - Run your program with valgrind: `valgrind ./prog`
  - More detailed output: `valgrind --leak-check=full ./prog`
- In general, you want to see `All heap blocks were freed -- no leaks are possible` in the output
- If there are memory leaks, you will see a summary of the number of bytes leaked and the number of blocks leaked
  - Use provided stack trace to identify where leaked memory was allocated
  - Warning: the leak may be somewhere else

Examining valgrind output: memory leak

```
$ valgrind --leak-check=full ./sort5 < grades
==3304551== Memcheck, a memory error detector
==3304551== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3304551== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==3304551== Command: ./sort5
==3304551==
65
76
85
88
98
100
==3304551==
==3304551== HEAP SUMMARY:
==3304551==     in use at exit: 64 bytes in 1 blocks
==3304551==   total heap usage: 5 allocs, 4 frees, 9,328 bytes allocated
==3304551==
==3304551== 64 bytes in 1 blocks are definitely lost in loss record 1 of 1
==3304551==    at 0x486F5F4: realloc (vg_replace_malloc.c:1801)
==3304551==    by 0x400913: grow (sort5.c:23)
==3304551==    by 0x400B8F: main (sort5.c:99)
==3304551==
==3304551== LEAK SUMMARY:
==3304551==    definitely lost: 64 bytes in 1 blocks
==3304551==    indirectly lost: 0 bytes in 0 blocks
==3304551==      possibly lost: 0 bytes in 0 blocks
==3304551==    still reachable: 0 bytes in 0 blocks
==3304551==         suppressed: 0 bytes in 0 blocks
==3304551==
==3304551== For lists of detected and suppressed errors, rerun with: -s
==3304551== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Examining valgrind output: invalid memory access

```
$ valgrind ./sort7 < grades
==3306902== Memcheck, a memory error detector
==3306902== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3306902== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==3306902== Command: ./sort7
==3306902==
65
76
85
88
98
100
==3306902== Invalid read of size 8
==3306902==    at 0x400C1C: main (sort7.c:115)
==3306902==  Address 0x4a46130 is 0 bytes inside a block of size 64 free'd
==3306902==    at 0x486B4AC: free (vg_replace_malloc.c:989)
==3306902==    by 0x400C17: main (sort7.c:112)
==3306902==  Block was alloc'd at
==3306902==    at 0x486F5F4: realloc (vg_replace_malloc.c:1801)
==3306902==    by 0x400913: grow (sort7.c:23)
==3306902==    by 0x400B8F: main (sort7.c:99)
==3306902==
65
==3306902==
==3306902== HEAP SUMMARY:
==3306902==     in use at exit: 0 bytes in 0 blocks
==3306902==   total heap usage: 5 allocs, 5 frees, 9,328 bytes allocated
==3306902==
==3306902== All heap blocks were freed -- no leaks are possible
==3306902==
==3306902== For lists of detected and suppressed errors, rerun with: -s
==3306902== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

The compiler can also be your friend

- Some compilers (e.g., gcc) have flags that can detect certain types of memory errors
  - `-fsanitize=address`: detects memory errors such as buffer overflows, use-after-free, etc.
  - `-fsanitize=undefined`: detects undefined behavior (e.g., integer overflow, invalid memory access)
  - Combined: `-fsanitize=address,undefined`
- Downside: Introduces additional overhead (slower execution) and may not catch all memory errors
- Nevertheless, can be useful as a first step in debugging memory errors

Overview

Summary

- Dynamic memory management is powerful but error-prone
- Common errors include memory leaks, dangling pointers, invalid free, double free and improper reallocation
- Tools like compiler sanitisers, meminfo and valgrind can help detect and debug memory errors
- Dates to remember:
  - Midterm: Wed, 3/4, 10:40–11:30am